

Relazione di Progetto “StudentHub”

Diego Andruccioli

Giovanni Morelli

Rei Mici

Anno Accademico 2025/2026

Sommario

StudentHub è un'applicazione Android per la gestione della carriera universitaria. Lo studente registra i propri esami (nome, voto, CFU, data), consulta statistiche sulla propria carriera accademica (media ponderata, media aritmetica, CFU totali, proiezione della base di laurea) e monitora i propri progressi attraverso un sistema di gamification: ogni esame superato genera punti esperienza (XP) calcolati dal server, contribuisce all'avanzamento di livello e può sbloccare badge tematici.

L'applicazione è sviluppata interamente in Kotlin adottando Clean Architecture multi-modulo, pattern MVVM con ViewModel e StateFlow, persistenza locale con Room e DataStore, comunicazione REST tramite Retrofit e OkHttp, background processing con WorkManager e interfaccia utente dichiarativa realizzata con Jetpack Compose.

La relazione è strutturata come segue: il Capitolo 1 presenta obiettivi e requisiti; il Capitolo 2 illustra le scelte progettuali e le librerie adottate; il Capitolo 3 descrive i dettagli implementativi più significativi con il formato Problema–Soluzione; il Capitolo 4 valuta punti forti e punti deboli del progetto; il Capitolo 5 propone miglie e sviluppi futuri.

Indice

Capitolo 1

Obiettivi e Requisiti

1.1 Descrizione del Progetto

StudentHub nasce dall'esigenza di offrire allo studente universitario uno strumento unico per la gestione della propria carriera accademica. Il dominio applicativo comprende due aree distinte ma interconnesse: la gestione del libretto esami (registrazione di voto, CFU e data per ciascun esame sostenuto, con calcolo automatico di media ponderata, media aritmetica, CFU totali accumulati e proiezione della base di laurea su 110) e la componente di gamification (assegnazione di punti XP a ogni esame superato secondo la formula $xp = voto \times cfu + (lode ? 50 : 0)$ calcolata lato server, avanzamento di livello, sblocco di badge per traguardi raggiunti e classifica degli studenti).

La logica di calcolo XP risiede esclusivamente nel backend Node.js, eliminando duplicazioni e garantendo che i dati mostrati sull'app siano sempre allineati con quelli ufficiali. L'applicazione è progettata per funzionare in modalità offline-first: le operazioni sul libretto sono disponibili anche senza connessione di rete e vengono sincronizzate con il server quando la connettività è ripristinata.

1.2 Requisiti Funzionali

- Registrarsi con nome, cognome, e-mail e password per creare un nuovo account.
- Accedere con e-mail e password; la sessione viene ricordata tra un'apertura e l'altra dell'app.
- Visualizzare il libretto con la lista di tutti gli esami registrati, ordinati per data.
- Aggiungere un esame specificando nome, voto (18–30 con possibilità di lode), CFU e data.
- Eliminare un esame dal libretto.
- Visualizzare le statistiche della propria carriera (media ponderata, CFU totali, base di laurea, grafico andamento voti).
- Monitorare il completamento degli obiettivi di gamification e i punti XP guadagnati.
- Personalizzare le impostazioni dell'account (tema di colorazione dei voti, soglie RGB) nella schermata Profilo.

- Effettuare il logout dalla schermata Profilo; la sessione viene invalidata sia localmente che sul server.
- Modificare un esame già inserito aggiornandone nome, voto, CFU o data.
- Ricevere una notifica locale quando la propria posizione in classifica cambia (sia in miglioramento che in peggioramento).

1.3 Requisiti Minimi del Corso

I requisiti minimi obbligatori del corso Sistemi Mobili sono soddisfatti dall'applicazione nei seguenti punti:

- **Kotlin puro:** l'intero progetto è scritto in Kotlin; non è presente alcun file Java.
- **Separazione in moduli:** il codice è organizzato nei moduli `:domain`, `:data`, `:ui` e `:app`, ciascuno con responsabilità distinte.
- **ViewModel e StateFlow:** ogni schermata utilizza un `ViewModel` dedicato come unico state holder; la UI osserva uno `StateFlow` tramite `collectAsStateWithLifecycle()` e non contiene logica di business.
- **Kotlin Coroutines:** tutte le operazioni di I/O (Room, chiamate di rete) avvengono su `Dispatchers.IO`; il Main Thread non viene mai bloccato.
- **Jetpack Compose:** l'interfaccia utente è implementata interamente in Jetpack Compose. Le liste di dati usano `LazyColumn` con chiave stabile per item, equivalente Compose di RecyclerView con `DiffUtil`.
- **Storage locale:** Room per la persistenza strutturata del libretto esami e dei dati di gamification; DataStore per le preferenze utente e i flag di onboarding.
- **Networking:** comunicazione REST con backend Node.js tramite Retrofit e OkHttp; autenticazione via cookie JWT HTTP-only con refresh automatico.
- **Background services:** WorkManager per la sincronizzazione offline degli esami e il controllo periodico del rank in classifica.
- **Runtime permission:** richiesta di `POST_NOTIFICATIONS` (Android 13+) con dialog di rationale contestuale prima dell'invio della notifica.
- **Relazione finale:** questo documento.

Capitolo 2

Scelte Progettuali

2.1 Architettura del Sistema

2.1.1 Clean Architecture Multi-Modulo

Il progetto è strutturato in quattro moduli Gradle distinti, ciascuno con responsabilità ben definite e dipendenze unidirezionali verso i layer inferiori.

Modulo `:domain` — nucleo dell'applicazione. Contiene i modelli di dominio (`Esame`, `User`, `Settings`, `GamificationStatus`), le interfacce dei repository (`EsameRepository`, `AuthRepository`, `GamificationRepository`, `SettingsRepository`) e i Use Cases, ciascuno con una sola responsabilità (`AddEsameUseCase`, `LoginUseCase`, `GetStatisticheUseCase`, ecc.). Il modulo è *puro Kotlin*: non importa nessuna classe Android, il che lo rende compilabile ed eseguibile in isolamento su qualunque JVM. Questo è il principale vantaggio pratico: i test unitari dei Use Cases e dei calcoli di business logic non richiedono un emulatore Android.

Modulo `:data` — implementa le astrazioni definite in `:domain`. Contiene le `@Entity` Room e i rispettivi `@Dao`, le implementazioni dei repository (`EsameRepositoryImpl`, `AuthRepositoryImpl`, ecc.), i client di rete (Retrofit API service, OkHttp con `PersistentCookieJar` e `TokenAuthenticator`), i `DataStore` per preferenze e sessione, e i `CoroutineWorker` di WorkManager. Nessuna Entity Room viene mai esposta al di fuori di questo modulo: i mapper traducono `Entity ↔ DomainModel` al confine del layer.

Modulo `:ui` — layer di presentazione realizzato interamente in Jetpack Compose. Contiene i composabile di schermata (`LibrettoScreen`, `StatisticheScreen`, `ObiettiviScreen`, `ProfiloScreen`, `LoginScreen`, `RegisterScreen`), i ViewModel e le sealed class di stato UI (`LibrettoUiState`, `ProfiloUiState`, ecc.). I ViewModel dipendono solo dalle interfacce del modulo `:domain`: non importano mai classi Room o Retrofit direttamente.

Modulo `:app` — punto di ingresso. Contiene `CustomApplication` che implementa l'interfaccia `RepositoryProvider` e istanzia manualmente tutti i repository, passando il `Context` necessario a Room e DataStore. È l'unico modulo che dipende contemporaneamente da tutti gli altri.

2.1.2 Schema delle Dipendenze tra Moduli

```
:app --> :ui, :domain, :data
:ui   --> :domain
```

```
:data --> :domain
:domain --> (nessuna dipendenza interna)
```

La freccia `:ui → :domain` e l'assenza di `:ui → :data` sono la scelta progettuale più rilevante: il layer UI non può mai accedere direttamente al database o al client di rete. Qualunque operazione sui dati deve passare obbligatoriamente per un Use Case. Questa regola è applicata strutturalmente dal sistema di build di Gradle: un tentativo di importare una classe Room da un composable produrrebbe un errore di compilazione.

La freccia `:data → :domain` significa che le implementazioni dei repository dipendono dalle interfacce definite nel dominio, non viceversa (Dependency Inversion Principle): sostituire Room con un'altra soluzione di persistenza richiederebbe modifiche solo in `:data`, lasciando intatti `:domain` e `:ui`.

2.1.3 Pattern MVVM e Flusso dei Dati

Il flusso di un'azione utente procede in senso discendente attraverso i layer:

```
Composable --(evento)--> ViewModel
    --> UseCase --> Repository (interfaccia)
                        |
                        RepositoryImpl --> Room DAO / Retrofit
```

Il flusso reattivo dei dati procede in senso ascendente:

```
Room DAO: Flow<List<EsameEntity>>
    --> RepositoryImpl: map + flowOn(Dispatchers.IO)
    --> UseCase: trasformazioni business
    --> ViewModel: stateIn(viewModelScope, Eagerly|WhileSubscribed)
                    --> StateFlow<UiState>
    --> Composable: collectAsStateWithLifecycle()
```

La UI non interroga mai attivamente i dati: osserva uno `StateFlow` e si ridisegna per recomposition ogni volta che lo stato cambia. Il `ViewModel` sopravvive ai cambi di configurazione (rotazione schermo, ridimensionamento finestra) grazie al meccanismo di Jetpack: la connessione al database Room non viene interrotta e lo stato dell'interfaccia viene preservato senza richiedere logica aggiuntiva.

2.2 Librerie e Tecnologie Utilizzate

2.2.1 Linguaggio e Ambiente di Sviluppo

- **Kotlin 2.2.10**: linguaggio ufficiale Android. Offre null safety, coroutine native, funzioni di estensione e data class rispetto a Java.
- **Android Studio**: IDE ufficiale con integrazione Gradle, debug su emulatore e profiler di memoria e CPU.
- **minSdk 31, targetSdk 36, Java 11 compatibility**: scelta che copre la grande maggioranza dei dispositivi Android attivi garantendo accesso alle API moderne (WorkManager, DataStore, Compose).

2.2.2 Gestione delle Dipendenze

- **Gradle Kotlin DSL**: build script type-safe rispetto a Groovy; gli errori di configurazione vengono rilevati a compile-time.
- **Version Catalog** (`libs.versions.toml`): gestione centralizzata delle versioni. Una singola modifica al catalog aggiorna tutte le dipendenze omogenee nell'intero progetto multi-modulo.
- **KSP 2.3.2 (Kotlin Symbol Processing)**: processore di annotazioni per Room. Scelto rispetto a KAPT per la velocità di compilazione incrementale significativamente superiore con Kotlin 2.x.

2.2.3 Jetpack e Librerie Core

- **Jetpack Compose (BOM 2025.12.00)**: toolkit UI dichiarativo. Elimina il file XML di layout, il ViewBinding e l'Adapter esplicito per le liste. Il componibile `NavigationSuiteScaffold` gestisce la navigazione adattiva (Bottom Bar su smartphone, Navigation Rail su tablet) senza codice condizionale.
- **Room 2.7.2**: ORM per SQLite. Scelto rispetto all'accesso diretto a SQLite per le query type-safe, l'integrazione nativa con `Flow` e la gestione strutturata delle migration. Usato per il libretto esami e la cache locale dei dati di gamification.
- **DataStore Preferences 1.1.1**: persistenza chiave-valore asincrona. Scelto rispetto a `SharedPreferences` perché le operazioni sono sospensive (nessun rischio ANR) e l'API è basata su `Flow`. Usato per il flag di sessione (`is_logged_in`), le preferenze utente (tema voti, soglie RGB) e il rank in cache per le notifiche.
- **ViewModel + StateFlow (Lifecycle 2.8.7)**: state holder MVVM. Il `ViewModel` sopravvive alla rotation; lo `StateFlow` espone uno snapshot immutabile dello stato corrente alla UI.
- **Kotlin Coroutines + Flow 1.9.0**: modello di concorrenza strutturato. Obbligatorio per il corso: ogni operazione I/O avviene su `Dispatchers.IO`, il Main Thread non viene mai bloccato.

2.2.4 Librerie Aggiuntive

- **Retrofit 2.11.0 + OkHttp 4.12.0**: client HTTP per la comunicazione REST con il backend Node.js. `OkHttp` gestisce il ciclo di vita dei cookie JWT tramite `PersistentCookieJar` e il rinnovo automatico del token tramite `TokenAuthenticator`. Scelti rispetto ad `URLConnection` per la leggibilità delle API Retrofit (interfacce Kotlin annotate) e per la ricchezza dell'ecosistema `OkHttp` (interceptor, authenticator, logging).
- **WorkManager 2.9.1**: scheduling garantito di task in background. Scelto rispetto ad `AlarmManager` perché gestisce automaticamente vincoli (connettività di rete), sopravvive ai riavvii del dispositivo e fornisce un'API Kotlin moderna con `CoroutineWorker`. Copre i requisiti d'esame per i background services.

- **Gson 2.11.0:** serializzazione JSON per la persistenza dei cookie OkHttp su `SharedPreferences`. Già presente come dipendenza transitiva di Retrofit; non aggiunge overhead al progetto.

Capitolo 3

Dettagli Implementativi

3.1 Gestione reattiva della lista esami

Problema La lista degli esami deve aggiornarsi automaticamente ogni volta che un esame viene aggiunto o eliminato, senza che la UI debba interrogare manualmente il database. Un'implementazione basata su query one-shot richiederebbe un refresh esplicito ad ogni operazione, con rischio di inconsistenze. È inoltre necessario evitare memory leak quando il Fragment viene distrutto in seguito a un cambio di configurazione.

Soluzione Il pattern adottato sfrutta la reattività nativa di Room tramite Flow. Il DAO espone fun `getAllEsami(): Flow<List<EsameEntity>>`: Room emette automaticamente una nuova lista ad ogni modifica della tabella `esami`. Il `RepositoryImpl` applica il mapper `toDomain()` e garantisce l'esecuzione su `Dispatchers.IO` tramite `flowOn`. Il `LibrettoViewModel` combina il Flow di Room con due `StateFlow` di ordinamento (`_sortBy`, `_sortOrder`) tramite l'operatore `combine()`, esponendo un unico `StateFlow<List<Esame>>` già ordinato. Ogni variazione in uno qualunque dei tre flussi sorgente innesca automaticamente un ricalcolo e una recomposition nella UI:

```
private val _sortBy      = MutableStateFlow(SortBy.DATA)
private val _sortOrder = MutableStateFlow(SortOrder.DESC)

val esami: StateFlow<List<Esame>> = combine(
    getEsamiUseCase(), _sortBy, _sortOrder
) { lista, by, order ->
    val sorted = when (by) {
        SortBy.DATA -> lista.sortedBy { it.dataEsame }
        SortBy.VOTO -> lista.sortedBy { it.voto }
        SortBy.CFU  -> lista.sortedBy { it.cfu }
    }
    if (order == SortOrder.DESC) sorted.reversed() else sorted
}.stateIn(viewModelScope, SharingStarted.Eagerly, emptyList())

init {
    // Carica le impostazioni dal server al login/reinstall: popola il
    //   ↳ DataStore
    // in modo che observeSettingsUseCase() emetta subito i valori reali
    // dell'account corrente (es. tema RGB) senza aspettare la schermata
    //   ↳ Profilo.
    viewModelScope.launch { getSettingsUseCase() }
}
```

```
fun refreshEsami() {  
    viewModelScope.launch { refreshEsamiUseCase() }  
}
```

Il blocco `init` non lancia più il caricamento degli esami: quel compito è delegato alla schermata tramite `LaunchedEffect(Unit)`, evitando una doppia chiamata concorrente a `GET /exams` che poteva causare una race condition nel pattern *check-then-insert* di `refreshEsami`. Il blocco `init` esegue invece `getSettingsUseCase()`: al momento del login (o dopo reinstallazione) il `SettingsDataStore` è vuoto — perché azzerato esplicitamente al logout tramite `SettingsDataStore.clear()` — e senza questa chiamata le impostazioni dell'account (es. il tema RGB per i colori dei voti) verrebbero applicate solo dopo la prima visita alla schermata Profilo.

Il caricamento degli esami avviene invece via `LaunchedEffect(Unit)` in `LibrettoScreen`:

```
LaunchedEffect(Unit) {  
    viewModel.refreshEsami()           // ri-fetch dal server al rientro in  
    ↪ composizione                     gamificationViewModel.refreshStats()  
}
```

Questo pattern — identico a quello adottato in `ObiettiviScreen` — garantisce che ogni volta che la schermata entra in composizione venga eseguita una chiamata server fresca. Room agisce da cache locale: se i dati sono già presenti la UI li mostra immediatamente, mentre la chiamata remota aggiorna eventualmente i dati in background.

Con l'adozione di Jetpack Compose, la UI osserva il `StateFlow` tramite `collectAsStateWithLifecycle()`, che integra automaticamente la consapevolezza del ciclo di vita: la raccolta si sospende quando la schermata non è visibile e riprende al rientro in foreground, prevenendo aggiornamenti inutili e memory leak. La lista degli esami è implementata con `LazyColumn` e chiave stabile per item (`key = { it.id }`): Compose usa la chiave per applicare la riconciliazione dell'albero delle composizioni, ottenendo lo stesso effetto di `DiffUtil.ItemCallback` senza richiedere un Adapter esplicito.

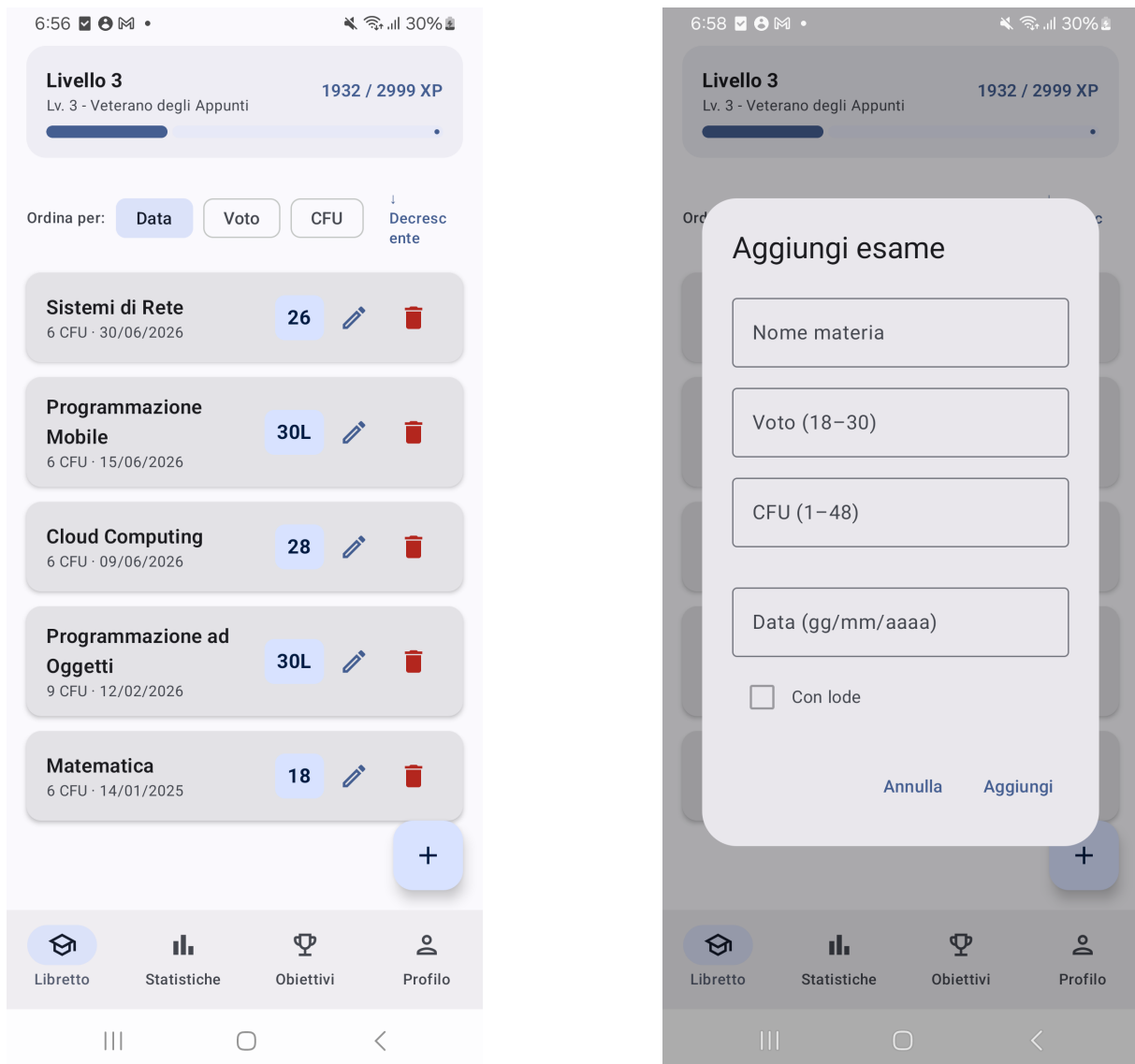


Figura 3.1: Schermata Libretto esami (**LibrettoScreen**): lista reattiva con ordinamento configurabile (sinistra) e dialog di aggiunta esame (**AddEsameDialog**) con validazione in linea (destra).

3.2 Architettura UI con Jetpack Compose e navigazione adattiva

Problema A partire dall'introduzione di Jetpack Compose nel corso, il layer UI deve adottare il paradigma dichiarativo. Occorre una struttura di navigazione che si adatti automaticamente a smartphone, tablet e finestre ridimensionabili su desktop, senza richiedere layout alternativi per ogni configurazione.

Soluzione Il modulo `:ui` è stato interamente convertito a Jetpack Compose. `MainActivity` estende `ComponentActivity` e delega tutta la UI a `setContent {}`, con `enableEdgeToEdge()` per il disegno sotto le barre di sistema.

Per la navigazione adattiva si utilizza `NavigationSuiteScaffold` della libreria `material3-adaptive-navigation-suite`: questo composabile seleziona automatica-

mente la modalità di navigazione più appropriata in base alle dimensioni della finestra — Bottom Navigation Bar su smartphone in verticale, Navigation Rail su tablet e in orizzontale, Navigation Drawer su schermi grandi. Il pattern segue esattamente quello insegnato nel corso e soddisfa il requisito di responsiveness senza alcun codice condizionale esplicito.

```
NavigationSuiteScaffold(  
  navigationSuiteItems = {  
    AppDestinations.entries.forEach { destination ->  
      item(  
        icon = { Icon(destination.icon, ...) },  
        label = { Text(destination.label) },  
        selected = destination == currentDestination,  
        onClick = { currentDestination = destination }  
      )  
    }  
  }  
) {  
  when (currentDestination) {  
    AppDestinations.LIBRETTO      -> LibrettoScreen(viewModel =  
      ↪ librettoViewModel)  
    AppDestinations.STATISTICHE  -> StatisticheScreen(viewModel =  
      ↪ statisticheViewModel)  
    AppDestinations.OBIETTIVI    -> ObiettiviScreen(viewModel =  
      ↪ obiettiviViewModel)  
    AppDestinations.PROFILO      -> ProfiloScreen(viewModel =  
      ↪ profiloViewModel)  
  }  
}
```

Tutti i ViewModel dell'applicazione utilizzano il pattern `ViewModelProvider.Factory` con dependency injection manuale: il composabile ospitante recupera i repository dal `RepositoryProvider` (implementato da `CustomApplication`) e li passa alla factory. Questo approccio migliora la testabilità — i ViewModel non dipendono da `Application` e possono essere istanziati in unit test con dipendenze fittizie — e garantisce un DI omogeneo nell'intero layer UI:

```
val profiloViewModel: ProfiloViewModel = viewModel(  
  factory = ProfiloViewModel.provideFactory(  
    getSettingsUseCase = GetSettingsUseCase(settingsRepository),  
    updateSettingsUseCase = UpdateSettingsUseCase(settingsRepository  
      ↪ ),  
    logoutUseCase = LogoutUseCase(authRepository)  
  )  
)
```

Ogni ViewModel sopravvive ai cambi di configurazione (rotazione schermo, ridimensionamento finestra su desktop) grazie al meccanismo standard di Jetpack.

3.3 Autenticazione e gestione della sessione utente

Problema L'applicazione comunica con un backend REST (Node.js/Express) che autentica tramite JWT in cookie HTTP-only. L'app deve: (1) presentare le schermate di login e registrazione finché l'utente non è autenticato; (2) persistere la sessione tra riav-

vii dell'app; (3) non bloccare il Main Thread durante le chiamate di rete; (4) tornare automaticamente al login in caso di sessione scaduta o assente.

Soluzione Il network layer è costruito su **Retrofit 2** + **OkHttp 4**. OkHttp è configurato con un **PersistentCookieJar** (SharedPreferences-backed, descritto nella Sezione ??), che persiste e reinvia automaticamente il cookie **token** a ogni richiesta. Il cookie è HTTP-only: non è mai accessibile in chiaro dal codice Kotlin.

La persistenza della sessione tra riavvii è affidata a **DataStore (Preferences)**: all'avvio il **AuthViewModel** legge il flag **is_logged_in** e lo espone come **StateFlow<Boolean?>** (**null** = caricamento in corso, **false** = non autenticato, **true** = autenticato):

```
val sessionState: StateFlow<Boolean?> = repository.isLoggedIn()
    .map { it as Boolean? }
    .stateIn(viewModelScope, SharingStarted.Eagerly, null)
```

Il composabile radice **RootNavigation** osserva **sessionState** e decide quale schermata mostrare senza alcuna navigazione imperativa:

```
when (sessionState) {
    null -> CircularProgressIndicator() // DataStore non ha ancora
        ↳ emesso
    false -> if (showRegister) RegisterScreen(...) else LoginScreen(...)
    true -> StudentHubApp()           // NavigationSuiteScaffold con
        ↳ le tab
}
```

Quando il backend restituisce una risposta di successo, **AuthRepositoryImpl** chiama **sessionDataStore.setLoggedIn(true)**: il **DataStore** emette il nuovo valore, il **StateFlow** si aggiorna e la UI transita automaticamente verso l'app senza alcuna chiamata esplicita alla navigazione.

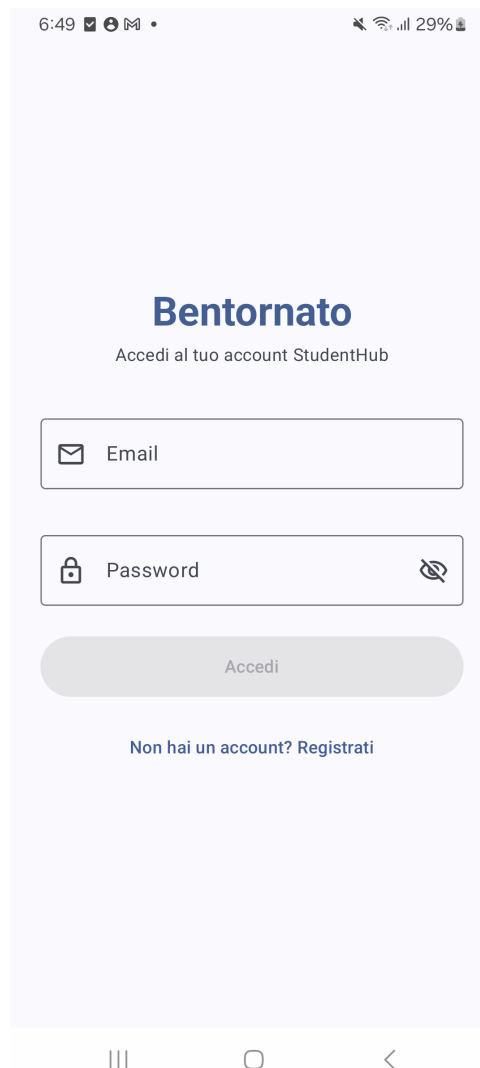


Figura 3.2: Schermata di accesso (**LoginScreen**): il pulsante e i campi si disabilitano durante la chiamata di rete; gli errori del server sono mostrati tramite **Snackbar** in italiano.

Il logout è implementato con lo stesso pattern dichiarativo. **LogoutUseCase** chiama `POST /api/auth/logout` avvolto in `runCatching`: anche se il backend non è raggiungibile, la sessione locale viene comunque invalidata. In particolare **AuthRepositoryImpl.logout()** esegue in sequenza: `NetworkClient.clearCookies()` (revoca cookie JWT), `db.clearAllTables()` (azzerà Room: esami, obiettivi, leaderboard), `RankDataStore.clear()` (azzerà XP/rank/livello), `SettingsDataStore.clear()` (azzerà le impostazioni cached) e `setLoggedIn(false)` (reindirizza alla login). Questo garantisce l'isolamento completo dei dati tra account diversi sullo stesso dispositivo: al successivo login, nessun dato dell'utente precedente sopravvive in locale. **ProfiloViewModel** espone `isLoggingOut: StateFlow<Boolean>` per disabilitare il pulsante e mostrare un indicatore di caricamento durante l'operazione asincrona. Al termine, quando il **DataStore** emette `false`, **RootNavigation** torna alla schermata di login automaticamente — nessuna navigazione imperativa richiesta.

3.4 Persistenza e rinnovo automatico del cookie JWT

Problema Il backend autentica le richieste tramite cookie HTTP-only: un access token (scadenza 15 minuti) e un refresh token (scadenza settimanale). Una `CookieJar` solo in memoria perde i cookie a ogni riavvio, forzando il login manuale. Alla scadenza dell'access token, senza gestione esplicita, tutte le chiamate API restituiscono 401.

Soluzione Il problema è risolto con due componenti nel `NetworkClient`. Il primo è `PersistentCookieJar`: implementazione di `CookieJar` che serializza i cookie su `SharedPreferences` (Gson) e li ricarica all'inizializzazione; le operazioni sono `@Synchronized` per la thread-safety di `OkHttp` e i cookie scaduti vengono filtrati in `loadForRequest`. Al logout, `clearCookies()` azzerava sia la cache in memoria sia le preferenze, impedendo il riutilizzo della sessione tra utenti diversi sullo stesso dispositivo.

Il secondo è `TokenAuthenticator`: `Authenticator OkHttp` che intercetta ogni risposta 401 ed esegue una chiamata sincrona a `POST /api/auth/refresh`. In caso di successo `OkHttp` ritenta la request originale in modo trasparente, senza che nessun `View-Model` o `Use Case` gestisca logica di retry. Se il refresh fallisce, invalida la sessione tramite `SessionDataStore.setLoggedIn(false)`: `RootNavigation` reagisce reattivamente e porta l'utente al login, senza navigazione imperativa. Una guardia esplicita previene loop infiniti: se la request fallita è già `/auth/refresh`, l'autenticator restituisce `null` immediatamente.

3.5 Sincronizzazione offline-first con WorkManager

Problema L'utente deve poter inserire e modificare esami anche senza connessione. Le operazioni offline non devono andare perse quando la rete torna disponibile, anche se l'app è stata chiusa nel frattempo.

Soluzione `EsameEntity` espone due flag di sincronizzazione: `pendingSync`: `Boolean` (default `true`) per add/update e `pendingDelete`: `Boolean` (default `false`) per i delete da ritentare. Quando la rete è disponibile, l'operazione viene inviata immediatamente tramite `Retrofit`; in caso di successo `markSynced()` azzerava `pendingSync` e salva il `remoteId`. Se un `DELETE` fallisce per assenza di rete, `deleteEsame` chiama `markPendingDelete(id)`: l'esame è nascosto dalla UI (`getAllEsami` filtra `pending_delete = 0`) ma rimane in `Room` per il retry. Analogamente, i record con `pendingSync = true` vengono esclusi dalla query `getPendingDeleteEsami`, e viceversa.

`SyncExamsWorker` esegue in ordine: (1) retry dei `DELETE` pendenti — se il server risponde 404 l'esame è già eliminato e viene rimosso localmente; (2) retry degli add/update pendenti con la stessa logica `POST/PUT`. Dopo ogni sync riuscito il worker aggiorna le statistiche di gamification e gli obiettivi. In caso di errori parziali restituisce `Result.retry()` per il backoff esponenziale di `WorkManager`.

La sincronizzazione del catalogo esami in entrata (`refreshEsami`) usa una strategia *last-write-wins* con priorità locale: gli esami modificati sul server vengono aggiornati in `Room` solo se la copia locale è già sincronizzata (`pendingSync = false`) e non è in attesa di eliminazione (`pendingDelete = false`). Se la copia locale è in modifica offline (`pendingSync = true`) la versione locale vince.

3.6 Notifiche di cambio rank con LeaderboardCheckWorker

Problema L'utente deve ricevere una notifica quando la sua posizione in classifica migliora, senza tenere l'app aperta. La notifica richiede il permesso `POST_NOTIFICATIONS` (Android 13+) da richiedere a runtime. Il worker deve confrontare il rank attuale con il precedente riutilizzando la logica di fetch già presente nel repository.

Soluzione `LeaderboardCheckWorker` (`CoroutineWorker`) è schedato in `CustomApplication` ogni 15 minuti con constraint `CONNECTED` e policy `KEEP` (nessuna schedulazione duplicata). Prima del fetch, il rank corrente viene letto da `RankDataStore`; `GamificationRepositoryImpl.getLeaderboard()` aggiorna `RankDataStore` come effetto collaterale tramite `saveMyRank()`. Il nuovo rank viene riletto e confrontato: se cambiato (in miglioramento o peggioramento), viene emessa una notifica con testo differenziato — “Rank migliorato!” con indicazione della nuova posizione se `newRank < oldRank`, “Rank cambiato” con la nuova posizione se `newRank > oldRank`. In caso di eccezione il worker restituisce `Result.retry()` per il backoff automatico di `WorkManager`. Il permesso `POST_NOTIFICATIONS` è richiesto in `ObiettiviScreen` con un dialog di rationale; l'utente può rifiutarlo senza compromettere le funzionalità principali dell'app.

3.7 Sincronizzazione dello stato di completamento degli obiettivi

Problema Il backend espone due endpoint distinti per la gamification: `GET /gamification/badges` restituisce il catalogo completo degli obiettivi con i soli campi anagrafici (`id`, `nome`, `descrizione`, `xp_valore`), senza alcun dato utente-specifico; `GET /gamification/my-badges` restituisce solo gli obiettivi effettivamente sbloccati dall'utente. Un'implementazione che interroghi unicamente il primo endpoint non è in grado di determinare quale obiettivo risulti completato: il campo `completato` è sempre assente dalla risposta del catalogo e Gson lo deserializza come `null`, rendendo ogni obiettivo permanentemente non completato nella UI anche dopo che il server ha assegnato i relativi punti XP.

Soluzione `ObiettivoRepositoryImpl.refreshObiettivi()` chiama entrambi gli endpoint in sequenza, costruisce un `Set<Int>` degli identificativi degli obiettivi sbloccati, quindi effettua il merge: per ogni voce del catalogo, `completato` viene impostato a `true` se e solo se il suo identificativo è presente nel set degli sbloccati.

```
override suspend fun refreshObiettivi(): Result<Unit> = withContext(
    ↪ ioDispatcher) {
    runCatching {
        val allBadges = api.getBadges().body() ?: emptyList()
        val myBadgesResponse = api.getMyBadges()
        val completedIds: Set<Int> = if (myBadgesResponse.isSuccessful)
            myBadgesResponse.body()?.map { it.safeId }?.toSet() ?:
            ↪ emptySet()
        else emptySet()
```

```

    val entities = allBadges.map { dto ->
        ObiettivoEntity(
            id = dto.safeId, nome = dto.nome,
            descrizione = dto.descrizione,
            completato = dto.safeId in completedIds,
            premioXp = dto.xpValore
        )
    }
    if (entities.isNotEmpty()) obiettivoDao.insertAll(entities)
}
}

```

Il pattern è identico a quello adottato nel frontend web (Vue3 + Pinia), dove `Promise.all` esegue le due fetch in parallelo e la computed property `badgesWithUnlockStatus` effettua lo stesso controllo di appartenenza al set. Poiché `refreshObiettivi()` viene invocato da `LibrettoViewModel` dopo ogni operazione CRUD sugli esami, il Flow esposto da Room aggiorna automaticamente la UI degli obiettivi senza alcuna navigazione o refresh manuale.

3.8 Architettura reattiva del layer gamification

Problema La schermata Obiettivi aggrega dati da sorgenti eterogenee: XP, livello e rank dell'utente (in `RankDataStore`), lista della classifica (cache Room) e lista degli obiettivi (Room). È necessario combinare queste sorgenti in un unico stato UI coerente, aggiornato reattivamente senza polling.

Soluzione `GamificationViewModel` usa `combine()` per fondere quattro Flow in un unico `StateFlow<GamificationUiState>`: `getGamificationDataUseCase()` (che combina sette chiavi di `RankDataStore` in uno `UserStats`), `getLeaderboardUseCase()` (Flow da Room), `_errorMessage` e `_isLoading`. Il metodo `refreshAll()` lancia in parallelo le due chiamate di rete tramite `async`, riducendo il tempo di attesa al massimo delle due:

```

val uiState: StateFlow<GamificationUiState> = combine(
    getGamificationDataUseCase(), // UserStats da RankDataStore
    getLeaderboardUseCase(),      // List<LeaderboardEntry> da Room
    _errorMessage,
    _isLoading
) { stats, leaderboard, error, isLoading ->
    when {
        error != null -> GamificationUiState.Error(error)
        isLoading -> GamificationUiState.Loading
        else -> GamificationUiState.Success(
            stats = stats,
            leaderboard = leaderboard.sortedByDescending { it.xpTotali }
        )
    }
}.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000),
    GamificationUiState.Loading)

```

Qualunque aggiornamento in una sorgente (es. un esame inserito che modifica XP o rank) si propaghi automaticamente alla UI senza refresh manuali.

3.9 Rendering dichiarativo e performante del grafico carriera

Problema La visualizzazione dell'andamento accademico (`StatisticheScreen`) richiede un grafico lineare capace di mostrare l'evoluzione dei voti nel tempo e la proiezione della media fissa. L'inclusione di librerie di charting di terze parti avrebbe appesantito inutilmente l'APK e infranto il paradigma puramente dichiarativo di Compose. Era inoltre necessario gestire le animazioni di ingresso senza causare pesantezza sul Main Thread e senza ricalcoli con alta potenza di calcolo delle coordinate spaziali ad ogni frame.

Soluzione Il grafico è stato implementato custom sfruttando le API di disegno nativo `Canvas` di Jetpack Compose. L'architettura del componente `CareerChart` si basa su un approccio a strati (Z-Order) pre-calcolato: 1. **Mappatura univoca:** I dati di dominio (`OffsetRelativo`) vengono mappati in coordinate pixel assolute una singola volta all'interno dello scope di disegno, garantendo coerenza topologica tra i punti dei voti e la curva di media ponderata. 2. **Animazione reattiva:** L'ingresso del grafico è governato da un `Animatable` che interpola un valore da 0f a 1f in 1500ms. Questo valore moltiplica i path vettoriali e l'opacità dei layer. 3. **Gestione dello strato visivo:** Il disegno avviene in ordine: prima la griglia di background, poi l'area del gradiente tramite `Brush.verticalGradient`, la curva creata dinamicamente interpolando i nodi con `cubicTo`, e infine il layer in foreground con le annotazioni.

```
// Animazione
val animationProgress = remember { Animatable(0f) }
LaunchedEffect(puntiVotiRelativi) {
    animationProgress.animateTo(1f, animationSpec = tween(durationMillis
        ↪ = 1500))
}

Canvas(modifier = modifier) {
    val progress = animationProgress.value
    // calcolo margini e coordinate

    // Area riempita con sfondo
    drawPath(
        path = fillPath,
        brush = Brush.verticalGradient(
            colors = listOf(errorColor.copy(alpha = 0.2f * progress),
                ↪ Color.Transparent),
            startY = pointsMedia.minOf { it.y },
            endY = height - margin
        )
    )
}
```

Questo approccio garantisce 60 FPS stabili durante l'animazione, poiché la recomposition non viene triggerata: la variazione di `animationProgress.value` invalida unicamente la fase di *Draw* del ciclo di vita di Compose, by-passando le fasi di *Composition* e *Layout*.



Figura 3.3: Schermata Statistiche (`StatisticheScreen`): scomposizione dei dati accademici e rendering custom su `Canvas` dell'andamento voti e media ponderata, con legenda.

3.10 Gestione dello stato durante operazioni asincrone nella UI

Problema Durante il salvataggio delle impostazioni nella `ProfiloScreen`, il `ViewModel` transita attraverso uno stato intermedio. Una prima implementazione usava uno stato `Saving` privo di dati e recuperava le impostazioni correnti accedendo direttamente a `viewModel.uiState.value` all'interno del composable. Questo approccio aggira il meccanismo reattivo di `Compose`: l'accesso diretto a `.value` non registra una dipendenza di recomposition e può restituire uno snapshot stantio se lo stato cambia tra due riesecuzioni della funzione composable.

Soluzione La sealed class `ProfiloUiState` è stata ristrutturata in modo che ogni stato porti con sé tutti i dati necessari alla UI, eliminando qualsiasi accesso diretto al flusso dello stato:

```
sealed class ProfiloUiState {  
    object Loading : ProfiloUiState()  
    data class Success(val settings: Settings) : ProfiloUiState()  
    data class Saving(val previousSettings: Settings) : ProfiloUiState()  
    data class Error(val message: String, val settings: Settings? = null  
        ↪ ) : ProfiloUiState()  
}
```

Quando l'utente avvia il salvataggio, il `ViewModel` transita a `Saving(settings)` portando le impostazioni correnti. Il composable mostra il form disabilitato con un `CircularProgressIndicator` nel bottone senza dover accedere ad altri `StateFlow`. Al termine dell'operazione il `ViewModel` emette `Success(settings)` (o `Error`): la UI si aggiorna per recomposition in modo interamente reattivo.

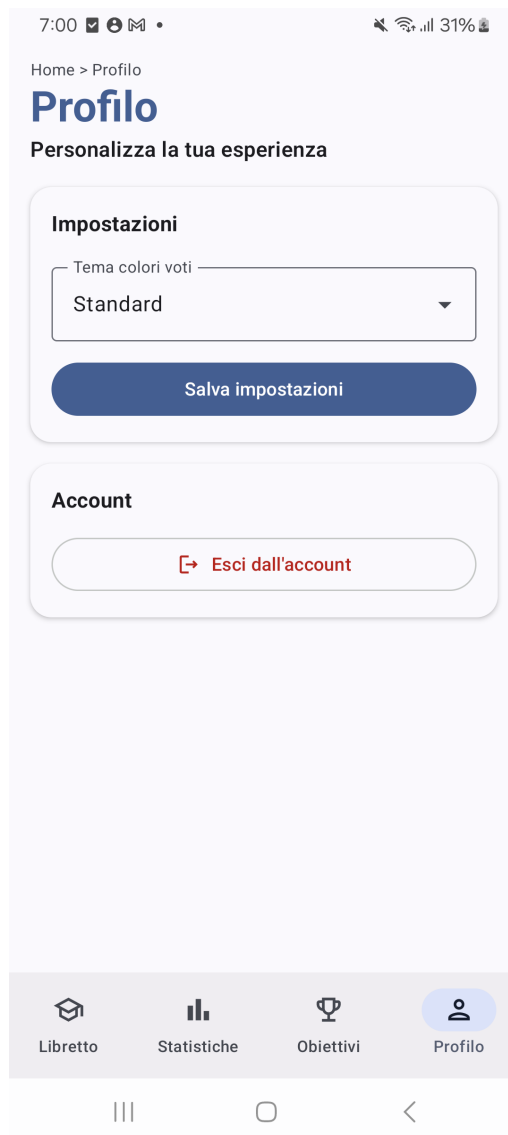


Figura 3.4: Schermata Profilo (`ProfiloScreen`): impostazioni tema voti, soglie RGB e pulsante logout. Il form viene disabilitato durante il salvataggio (`ProfiloUiState.Saving`) per prevenire modifiche concorrenti.

3.11 Test unitari del layer di dominio

Problema La correttezza dei calcoli statistici (`GetStatisticheUseCase`) non può essere verificata solo con test manuali sull'app. È necessario garantire che le trasformazioni sui dati producano risultati precisi (media ponderata, base di laurea, andamento progressivo) per tutti i casi limite, e che i `ViewModel` gestiscano correttamente gli stati di errore e i percorsi CRUD.

Soluzione Il modulo `:domain` non ha dipendenze Android: le classi di business logic sono pure funzioni Kotlin testabili con `JUnit4` e `kotlinx-coroutines-test` senza emulatore. I `ViewModel` sono stati migrati da `AndroidViewModel` a plain `ViewModel` con factory esplicita, eliminando la dipendenza da `Application` e rendendo possibile l'istan-

ziazione diretta nei test con dipendenze fittizie. Sono stati scritti 29 test unitari suddivisi in cinque suite:

- **GetStatisticheUseCaseTest** (6 test): verifica media ponderata, CFU totali, base di laurea e andamento progressivo. Usa un repository fittizio basato su `flowOf()`, eliminando qualsiasi dipendenza da Room.
- **AuthUseCaseTest** (7 test), **LibrettoViewModelTest** (7 test), **Statistiche-ViewModelTest** (4 test), **ProfiloViewModelTest** (5 test): copertura delle operazioni CRUD e dei percorsi di errore nei ViewModel principali, con repository mock forniti da Mockito-Kotlin.

`UnconfinedTestDispatcher` sostituisce `Dispatchers.Default` per rendere i test con coroutine deterministici e privi di `delay` artificiali.

3.12 Deserializzazione della data esame dal backend

Problema Il campo `data` restituito dall'endpoint `GET /api/exams` non è una stringa `DATE` pura (`"2026-05-25"`) ma una stringa ISO 8601 completa (`"2026-05-25T00:00:00.000Z"`), prodotta dalla libreria `mysql2` di Node.js quando la colonna MySQL è di tipo `DATE` e l'opzione `dateStrings: true` non è attiva. La chiamata `LocalDate.parse(dto.data)` di Java Time solleva una `DateTimeParseException` perché il formatter predefinito si aspetta esattamente dieci caratteri nel formato `yyyy-MM-dd`. L'eccezione era inghiottita da un `runCatching` in `refreshEsami()`: Room restava vuota senza alcun messaggio di errore visibile, rendendo il bug particolarmente difficile da diagnosticare.

Soluzione Poiché il backend non è modificabile, la correzione è applicata lato Android:

```
val remoteDate = LocalDate.parse(dto.data.take(10))
```

`String.take(10)` estrae i primi dieci caratteri (`"2026-05-25"`) prima del parsing, eliminando il suffisso temporale indipendentemente dal formato esatto prodotto dal backend. L'operazione è sicura anche nel caso in cui il backend venisse corretto in futuro e tornasse a inviare solo la data: `take(10)` su una stringa di dieci caratteri restituisce la stringa invariata.

3.13 Allineamento dei valori ENUM `tema_voti` tra Android e MySQL

Problema La colonna `tema_voti` nel database MySQL è definita come `ENUM('DEFAULT', 'RGB')`. L'app Android inviava i valori `"STANDARD"` e `"LODE"` non presenti nell'ENUM: MySQL in strict mode rifiuta l'inserimento con un errore 500, che compariva in UI come *"salvataggio impostazioni fallito"*. Parallelamente, la strategia di aggiornamento era difettosa: `updateSettings()` inviava prima la richiesta al server e aggiornava il `DataStore` solo in caso di successo. Ciò significava che un errore di rete transitorio annullava la preferenza locale dell'utente, anche se l'app funzionava correttamente offline.

Soluzione Sono stati applicati due interventi distinti. Il primo allinea i valori inviati all'ENUM del backend: i valori selezionabili nella `ProfiloScreen` sono ora "DEFAULT" e "RGB"; la voce "LODE" è rimossa dal dropdown e dai valori di fallback in tutti i `ViewModel` e nei test. Il secondo corregge la strategia di persistenza in `SettingsRepositoryImpl`: il `DataStore` viene aggiornato *prima* della chiamata API; la chiamata al server è effettuata in un `runCatching` separato il cui esito viene ignorato. In questo modo l'utente vede immediatamente il risultato del salvataggio locale, e la sincronizzazione remota avviene in modo best-effort senza bloccare l'operazione. Questa strategia rispecchia il principio offline-first già adottato per il libretto esami.

3.14 Deserializzazione del campo lode dal backend

Problema MySQL memorizza il campo `lode` come `TINYINT(1)`. La libreria `Node.js mysql2` serializza questo tipo come numero intero (0 o 1) anziché come valore booleano JSON (`false/true`). Il DTO Android `ExamDto.lode` era dichiarato come `Boolean`: `Gson` lancia una `IllegalStateException` (*"Expected a boolean but was NUMBER"*) durante la deserializzazione di ogni esame. L'eccezione veniva inghiottita silenziosamente dal blocco `runCatching` in `refreshEsami()`, impedendo completamente il popolamento di `Room` con i dati del server — lo stesso punto di fallimento del bug sulla data ISO 8601, ma con una causa diversa.

Soluzione Il tipo del campo `lode` in `ExamDto` è stato modificato da `Boolean` a `Int`. Nei punti del codice in cui il DTO viene mappato a `EsameEntity` (inserimento e aggiornamento in `refreshEsami()`), la conversione esplicita `dto.lode != 0` produce il valore booleano atteso. `ExamRequest.lode` rimane `Boolean`: `Gson` lo serializza correttamente come `true/false` e lo schema `Zod` del backend accetta valori booleani in ingresso.

3.15 Visibilità degli errori silenti nel layer di rete

Problema Il layer dati conteneva circa dieci blocchi `runCatching` che inghiottivano ogni eccezione senza alcun log. Tre bug distinti (data ISO 8601, ENUM `tema_voti`, `lode TINYINT`) erano rimasti latenti per settimane perché nessun messaggio di errore appariva nel `Logcat`. La diagnosi richiedeva di collegare un debugger e impostare breakpoint manualmente, rendendo il ciclo di debug molto lento.

Soluzione Aggiunto il callback `.onFailure { e -> Log.e(TAG, "...", e) }` a ogni blocco `runCatching` critico nei repository (`EsameRepositoryImpl`, `ObiettivoRepositoryImpl`, `SettingsRepositoryImpl`). Il tag per ogni repository è una costante `private const val TAG` dedicata: filtrare il `Logcat` per `EsameRepository`, `ObiettivoRepository` o `SettingsRepository` mostra immediatamente ogni chiamata API con il suo esito. Per il traffico HTTP raw (header, body, status code) è già attivo `HttpLoggingInterceptor.Level.BODY` in `NetworkClient`: filtrare per tag `OkHttp` nel `Logcat` espone l'intera comunicazione con il server senza alcuna modifica al codice.

3.16 Risposta HTTP non vuota su endpoint che non restituiscono dati

Problema Due endpoint del backend — DELETE /exams/:id e POST /auth/logout — rispondono con un corpo JSON di conferma ({`"message": "..."`}) anziché un corpo vuoto. In Retrofit 2, dichiarare il tipo di ritorno come `Response<Unit>` fa sì che il convertitore Gson tenti di deserializzare quel corpo come `Unit`; su corpi non vuoti questo produce un'eccezione a runtime.

Per il DELETE l'impatto era funzionale: il blocco `runCatching` nel metodo `deleteEsame()` di `EsameRepositoryImpl` catturava l'eccezione e restituiva `null`; il controllo di successo della response risultava quindi `false`. L'app chiamava `markPendingDelete` invece di eliminare la riga da Room, lasciando l'entità con `pendingDelete = true` anche se il server aveva già eliminato l'esame. L'esame spariva dalla UI (filtro `pending_delete = 0`), ma `SyncExamsWorker` doveva eseguire un ciclo di retry aggiuntivo.

Per il logout l'impatto era nullo dal punto di vista dell'utente: l'eccezione veniva comunque inghiottita da `runCatching` e il codice successivo (`clearCookies`, `clearAllTables`, `setLoggedIn(false)`) veniva eseguito regolarmente. Tuttavia la chiamata POST /auth/logout non veniva completata correttamente lato OkHttp (connessione non rilasciata).

Soluzione Entrambi i metodi sono stati cambiati da `Response<Unit>` a `Response<ResponseBody>`, in analogia con il metodo `updateSettings()` già presente in `SettingsApiService`. Il corpo viene chiuso esplicitamente con `body()?.close()` per rilasciare la connessione OkHttp; nessun campo del corpo viene letto poiché non è necessario.

```
// AuthApiService.kt
@POST("auth/logout")
suspend fun logout(): Response<ResponseBody>

// ExamApiService.kt
@DELETE("exams/{id}")
suspend fun deleteEsame(@Path("id") id: Int): Response<ResponseBody>

// AuthRepositoryImpl.kt
runCatching { apiService.logout() }.getOrNull()?.body()?.close()

// EsameRepositoryImpl.kt
if (response?.isSuccessful == true) {
    response.body()?.close()
    dao.deleteEsame(entity)
}
```

3.17 Allineamento dei nomi di campo nelle risposte di aggiunta e modifica esame

Problema Le risposte JSON del backend per POST /exams e PUT /exams/:id usano la convenzione *snake_case* per i campi opzionali (`xp_totali_guadagnati`,

`nuovi_badge`, `badge_revocati`, `xp_difference`), mentre i corrispondenti data class Kotlin (`AddExamResponse`, `UpdateExamResponse`) usavano i nomi camelCase senza annotazioni `@SerializedName`. Gson non trovava corrispondenza e popolava sempre quei campi con `null`, impedendo a potenziali consumatori futuri di leggere i dati restituiti dal server.

Soluzione Aggiunte le annotazioni `@SerializedName` ai campi coinvolti, allineando la deserializzazione Gson al contratto del backend:

```
data class AddExamResponse(  
    val ids: List<Int>,  
    @SerializedName("xp_totali_guadagnati") val totalXp: Int?,  
    @SerializedName("nuovi_badge") val newBadges: List<Any>?  
)  
  
data class UpdateExamResponse(  
    @SerializedName("nuovi_badge") val newBadges: List<Any>?,  
    @SerializedName("badge_revocati") val revokedBadgeIds: List<Any>?,  
    @SerializedName("xp_difference") val xpDifference: Int?  
)
```

3.18 Sincronizzazione unidirezionale delle impostazioni utente

Problema Le impostazioni utente (tema voti, soglie RGB) erano sincronizzate correttamente nella direzione mobile → web: la funzione `saveSettings()` invocava `updateSettingsUseCase()`, che aggiornava il `DataStore` locale e chiamava `PUT /settings` sul backend. La direzione opposta, web → mobile, era però interrotta: `getSettingsUseCase()` veniva chiamato una sola volta al momento della creazione del `ProfiloViewModel` (nel blocco `init {}`), e `observeSettingsUseCase()` osservava esclusivamente il `DataStore` locale. Di conseguenza, qualsiasi modifica effettuata tramite l'interfaccia web non veniva mai recepita sull'app mobile finché l'utente non eseguiva `logout` e `ri-login`, oppure non forzava la chiusura dell'app.

Soluzione Il pattern già adottato in `LibrettoScreen` — un `LaunchedEffect(Unit)` che recupera i dati dal server ad ogni ingresso nella schermata — è stato applicato anche a `ProfiloScreen`. Ogni volta che l'utente naviga alla schermata Profilo, il blocco esegue `viewModel.loadSettings()`, che interroga il backend e aggiorna il `DataStore`; `observeSettingsUseCase()` reagisce automaticamente all'aggiornamento e la UI mostra i valori più recenti.

```
// ProfiloScreen.kt  
LaunchedEffect(Unit) { viewModel.loadSettings() }
```

La chiamata è idempotente e ha un costo trascurabile: il backend restituisce un payload JSON minimo e il `DataStore` viene aggiornato solo se i valori sono cambiati. Il `loadSettings()` nell'`init {}` del `ViewModel` è stato mantenuto per garantire il caricamento immediato alla prima apertura dell'app, prima che la composizione della schermata raggiunga il `LaunchedEffect`.

Capitolo 4

Punti Forti e Punti Deboli

4.1 Punti Forti

- **Modulo `:domain` puro Kotlin:** nessuna dipendenza Android nel layer di business logic. I Use Cases e i calcoli statistici sono testabili su qualunque JVM senza emulatore, con tempi di esecuzione del test suite inferiori ai 5 secondi.
- **UI interamente reattiva:** la UI non interroga mai i dati in modo attivo. Ogni aggiornamento (nuovo esame, cambio XP, modifica impostazioni) viene propagato automaticamente da Room tramite Flow fino al composabile, senza polling né notifiche manuali.
- **Strategia `offline-first` robusta:** i flag `pendingSync` e `pendingDelete` su `EsameEntity` coprono l'intero ciclo di vita CRUD anche senza connessione. Le eliminazioni fallite vengono nascoste dalla UI (`getAllEsami` filtra `pending_delete = 0`) e ritentate da `SyncExamsWorker` al ritorno della rete, anche con l'app chiusa.
- **Token refresh trasparente:** `TokenAuthenticator` gestisce il rinnovo del JWT senza che nessun ViewModel o Use Case ne sia consapevole. Il codice applicativo non contiene mai logica di retry per gli errori 401.
- **Navigazione adattiva senza codice condizionale:** `NavigationSuiteScaffold` adatta automaticamente la navigazione al form factor (smartphone, tablet, desktop windowing) senza un singolo if sul tipo di schermo.
- **Factory pattern uniforme sui ViewModel:** tutti i ViewModel usano `ViewModelProvider.Factory` con DI manuale, garantendo testabilità omogenea e coerenza con il pattern insegnato dal corso.

4.2 Punti Deboli

- **DI manuale non scalabile:** il pattern `RepositoryProvider` funziona per il perimetro attuale, ma con l'aggiunta di nuovi repository o dipendenze trasversali diventa verboso. Hilt sarebbe preferibile su un progetto di dimensioni maggiori.
- **Test di integrazione assenti:** Room DAO e `CoroutineWorker` non sono coperti da test automatici (richiedono rispettivamente un database in-memory e l'infrastrut-

tura `WorkManagerTestInitHelper`). La copertura del test suite si ferma al layer domain e `ViewModel`.

- **Gestione errori di rete non granulare:** gli errori di rete (`IOException`), di timeout e di risposta server (4xx/5xx) vengono mappati sulla stessa classe `Error` nella UI. Una gerarchia di errori distinta permetterebbe messaggi più precisi all'utente.
- **SharingStarted.Eagerly in `LibrettoViewModel`:** solo `LibrettoViewModel` usa `Eagerly` — scelta deliberata per mantenere il Flow di Room sempre attivo e garantire la disponibilità immediata della lista al rientro in composizione. Gli altri `ViewModel` (`GamificationViewModel`, `StatisticheViewModel`, `ObiettiviViewModel`) adottano `WhileSubscribed(5000)`, che sospende l'upstream dopo cinque secondi senza sottoscrittori, ottimizzando le risorse. Il trade-off è accettabile per le dimensioni del progetto attuale.

Capitolo 5

Migliorie e Sviluppi Futuri

5.1 Migliorie a Breve Termine

- **Test di integrazione per Room:** aggiungere test con database in-memory per verificare query DAO, migration e comportamento del `Flow` al variare dei dati persistiti.
- **Gerarchia di errori di rete:** distinguere `NetworkError` (assenza di connessione), `TimeoutError` e `ServerError` per mostrare messaggi contestuali e strategie di retry differenziate nella UI.
- **Accessibilità:** completare i `contentDescription` su tutti gli elementi interattivi dei composables per il supporto agli screen reader.
- **Indicizzazione Room:** aggiungere `@Index` sulle colonne `remote_id`, `pending_sync` e `pending_delete` di `EsameEntity`, utilizzate frequentemente dal `SyncExamsWorker` per le query di filtering.

5.2 Sviluppi Futuri

- **Notifiche Push via FCM:** sostituire le notifiche locali del worker `LeaderboardCheckWorker` con push server-side, eliminando il polling periodico dal dispositivo.
- **Widget home screen:** visualizzazione rapida di media ponderata e livello XP senza aprire l'app, tramite Glance API (Jetpack Compose per widget).
- **Importazione piano di studi:** integrazione con le API di Esse3 o Almaesami per pre-popolare il libretto con gli esami del piano di studi ufficiale.
- **Dependency Injection con Hilt:** migrazione da `RepositoryProvider` manuale a Hilt per migliorare la scalabilità del progetto e ridurre il boilerplate nelle factory dei `ViewModel`.